

Real-Time Notification & Event-Driven Architecture Proposal

1. Executive Summary & Architecture Objectives

At BookMyShow's scale, millions of movie ticket bookings, payment confirmations, and promotional events occur concurrently. Tightly coupling downstream notification delivery (SMS, email, push notifications) to the synchronous booking API introduces unacceptable checkout latency, thread pool exhaustion, and vulnerability to third-party gateway downtime.

This proposal establishes a fully decoupled, **event-driven publish/subscribe architecture** using Apache Kafka. By isolating event production from delivery workers, the system guarantees sub-second booking checkout responses while reliably fanning out notifications, delayed show reminders, and real-time dashboard updates.

2. End-to-End Event Lifecycle

The lifecycle of an event flows through four deterministic stages:

1. **Emission:** When a state transition occurs (e.g., payment verified), `BookingService` publishes a JSON `BookingConfirmedEvent` to Kafka topic `booking-events` using `bookingId` as the partition key.
2. **Ingestion & Fan-Out:** Kafka broadcasts the event in parallel to independent consumer groups (`NotificationService`, `AnalyticsService`, `LoyaltyService`) without inter-service dependencies.
3. **Real-Time Channel Dispatch:** `NotificationService` resolves recipient contact preferences and delegates tasks to isolated worker pools (Email, SMS, Push, WebSocket).
4. **Delayed Scheduling:** For time-shifted events, `ReminderService` schedules tasks in a distributed delay queue (Redis ZSET), firing `ShowReminderEvent` exactly 2 hours prior to showtime.

3. Delivery Semantics & Idempotency Rationale

We implement **At-Least-Once Delivery with Idempotent Consumer Processing**:

- **At-Most-Once:** Rejected because dropped network packets would lead to unissued tickets and severe customer dissatisfaction.
- **Strict Exactly-Once (Two-Phase Commit):** Rejected because distributed transactions across Kafka and external third-party communication gateways introduce severe latency and lock contention.
- **At-Least-Once + Idempotency:** Selected as the production standard. Kafka guarantees delivery. Upon message receipt, workers verify the unique `eventId` against Redis before invoking external gateways. If already processed, the duplicate message is acknowledged and skipped, achieving pragmatic exactly-once execution.

4. Architecture Summary Matrix

Subsystem	Technology	Delivery Guarantee	Primary Responsibility	Failure Fallback
Event Ingestion	Apache Kafka	At-least-once (acks=all)	Partitioned event streaming across producers and consumers	In-memory producer buffering with exponential backoff
Idempotency Store	Redis Cluster	Strongly consistent	Prevents duplicate SMS/Email dispatch using 24-hour eventId TTL	Local worker in-memory LRU fallback cache
Delayed Scheduler	Redis Sorted Sets	Persistent ZSET	Schedules show reminders; workers poll due tasks every 5 seconds	Consumer offset rewind & manual scheduler trigger
Multi-Channel Delivery	Worker Pools (SES, Twilio, FCM)	Idempotent dispatch	Delivers rich HTML emails, SMS codes, push alerts, and WebSockets	3 exponential retries, then route to notifications-dlq

5. Resilience, Fault Recovery & Disaster Mitigation

- **Broker Resilience:** Topics are configured with a replication factor of 3 and `min.insync.replicas = 2`. The failure of an individual broker does not halt publishing or cause message loss.
- **Dead-Letter Queue Isolation:** Unresolvable errors (e.g., invalid phone numbers, external gateway 5xx errors) route directly to `notifications-dlq`, preventing poison-pill messages from blocking consumer topic partitions.
- **Provider Redundancy:** Outbound SMS and email workers incorporate automated failover routing (e.g., switching from Twilio to AWS SNS if failure rates exceed 20%).
- **Historical Event Replay:** Leveraging Kafka's 7-day retention log, consumer offsets can be rewound to a specific UTC timestamp to replay missed events following extended third-party outages.

6. Observability, Operational Readiness & SLOs

- **Distributed Tracing:** Every request carries an origin `eventId` and `userId` propagated through message headers, enabling end-to-end tracing across ELK dashboards.
- **Core SLO Enforcement:**
 - **Booking Confirmation Latency:** 99% of ticket confirmations delivered within **< 5.0 seconds** of payment completion.
 - **Show Reminder Timeliness:** 95% of show reminders dispatched within **< 1.0 minute** of the scheduled 2-hour advance trigger.